

Tool-Trace Compaction

Beş Ajanlı Teknik Referans — Hermes · OpenClaw · OpenCode · Codex · Claude Code

Derleme tarihi: 2026-08-08 · Kaynak: her sistemin kaynak kodundan sökülmüş teknik dökümanlar

Bu doküman nedir?

Bir ajan, tool çağırdıkça (dosya okuma, komut çıktısı, web getirme...) bu çağrılarının **izi (trace)** context penceresini şişirir. **Tool-trace compaction**, bu izi — mesaj bütünlüğünü ve tool_call/tool_result eşleşmesini bozmadan — küçültme sanatıdır. Bu referans, beş üretim ajanının bunu **tam olarak nasıl** yaptığını, gerçek fonksiyon adları, sabitler ve adım-adım örneklerle bir araya getirir.

İki ekol

- Deterministik, LLM'siz sıkıştırma** — kurallarla buda/kes/özetle; ucuz, öngörülebilir, prompt-cache dostu. Örnek: **Hermes** (4 geçiş), **OpenCode** (prune), **Codex** (ortadan-kesme).
- LLM ile chunk-özetleme** — eski izi bir modele verip özet üretir; daha akıllı ama maliyetli, cache-kıran. Örnek: **OpenClaw**, **Codex** (model-turn compaction), **Claude Code** (auto-compaction).

Çoğu sistem ikisini **katmanlı** kullanır: önce ucuz deterministik kat, yetmezse LLM katı.

İçindekiler

Bölüm	Ekol / yaklaşım
1. Hermes	Deterministik 4-geçiş (LLM'siz)
2. OpenClaw	LLM chunk-özetleme (12 adım)
3. OpenCode	2 katman: canlı spill + deterministik prune
4. Codex	2 katman: ortadan-kesme + model-turn windowing
5. Claude Code	Microcompaction + auto-compaction (gözlem)

Not: Mermaid diyagramları PDF'te kaynak (text) olarak gösterilir. Bölüm başlıkları PDF yer-imini (bookmark) panelinden de gezilebilir.

Hermes-Agent — Tool-Trace Compaction

(Yalnızca Tool İzi): Her Detay

Kapsam: Bu belge **sadece tool-trace compaction**'ı anlatır — yani tool çağrılarını ve sonuçlarının context içindeki izini nasıl küçülttüğünü. Konuşma-özetleme (middle-turn LLM summarization), eşik hesabı, anti-thrash gibi *genel context compaction* konuları kapsam dışıdır; onlar ayrı bir mekanizmadır. Burada odak: `_prune_old_tool_results` ve `prune_tool_results_only` — Hermes'in LLM'siz, deterministik, tool-sonucu budama motoru.

Kaynak: [../harnesses/hermes-agent/agent/context_compressor.py](https://github.com/ai-ware/harnesses/hermes-agent/agent/context_compressor.py). Her satır koddan doğrulanmıştır; issue numaraları (#61932, #32106...) koddaki yorumlardan gelir.

0. Tool-trace compaction tam olarak nedir (ve ne DEĞİLDİR)

Tool-trace = context içindeki tool çağrılarının (`assistant` mesajındaki `tool_calls`) ve tool sonuçlarının (`role: "tool"` mesajları) toplamı. Bir ajan çalışıkça bunlar birikir ve context'in en büyük şişme kaynağı olur.

Tool-trace compaction = bu tool mesajlarını, **konuşmanın anlamını LLM'e sormadan**, deterministik kurallarla küçültmek. Üç işlem yapar:

- Dedup** — aynı sonucu tekrar eden tool mesajlarını tekilleştir.
- Informative summary** — büyük tool sonuçlarını bilgi taşıyan tek satıra indir.
- Argument truncation** — büyük tool çağrı argümanlarını kısalt.

Ne DEĞİL: Bu bir LLM özetlemesi değildir. "Orta turn'leri auxiliary model'e verip özet yazdırma" tamamen ayrı bir faz (full compression'ın Adım 4'ü). Tool-trace budama **hiç model çağırılmaz** — bu yüzden ucuz, hızlı ve "kalite-riski sıfır"dır.

İki giriş noktası, aynı çekirdek:

Fonksiyon	Ne zaman	LLM özeti	Tetik
<code>_prune_old_tool_results</code>	Full compression'ın Adım 1'i olarak	— (sadece deterministik)	Eşik dolunca
<code>prune_tool_results_only</code>	Bağımsız, maliyet için	Yok (sadece bu 3 pass)	<code>proactive_prune_tokens</code>

İkisi de **aynı 3 pass'i** çalıştırır; fark tetik ve tail-koruma stratejisidir.

1. Mesaj yapısı — neyi bozamayız (kritik ön bilgi)

Tool-trace, `messages[]` dizisinde şöyle görünür:

```
{"role": "assistant", "tool_calls": [  
  {"id": "call_7", "function": {"name": "read_file", "arguments": "{\"path\": \"a.py\"}"}}  
]}  
{"role": "tool", "tool_call_id": "call_7", "content": "<dosyanın 40.000 karakteri>"}
```

Değişmez kural: Her `tool` mesajı, `tool_call_id` ile bir `assistant.tool_calls[].id`'ye eşleşmek zorundadır. Bu çift bozulursa (ör. `tool` mesajını silersen ama çağırıcı bırakırsan) sağlayıcı **HTTP 400** döner. Bu yüzden budama:

- Tool sonucunun **içeriğini** değiştirir, **mesajın kendisini silmez** (`tool_call_id` korunur).
- Argümanları kısaltırken bile **geçerli JSON** bırakır (yoksa 400).

Budama en başta bir **indeks** kurar — her `tool_call_id`'yi `tool` adına + argümanlarına bağlar (böylece bir sonucu görünce onu üreten çağırıcı bilir):

```
call_id_to_tool: Dict[str, tuple] = {} # "call_7" -> ("read_file", "\\path\\:\\a.py\\")
```

Bu indeks, "informatif özet"in nasıl `[read_file] read a.py ...` üretebildiğinin sırrıdır: sonucun içeriği gitse de, onu *hangi çağırıcının* ürettiği bu indekste durur.

2. Prune boundary — nereye kadar budanır?

Budama, **son mesajları korur** (yakın bağlam dokunulmaz) ve sınırdan öncekini budar. Sınır iki yolla belirlenir:

(a) Token bütçesiyle (öncelikli)

`protect_tail_tokens` verilmişse, sondan geriye doğru yürünür, token biriktirilir; bütçe aşıldıkça sınır çizilir.

```
accumulated = 0
for i in range(len(result)-1, -1, -1):
    msg_tokens = _estimate_msg_budget_tokens(msg)
    if accumulated + msg_tokens > protect_tail_tokens and (len(result)-i) >= min_protect:
        boundary = i; break
    accumulated += msg_tokens
```

(b) Mesaj sayısı (yedek)

`protect_tail_tokens` yoksa: `prune_boundary = len(result) - protect_tail_count`.

Taban: `_MAX_TAIL_MESSAGE_FLOOR = 8`

Token bütçesi kullanılsa bile en az `min(protect_tail_count, len, 8)` mesaj korunur. **Neden 8, 20 değil?** Koddaki yorum: *"varsayılan `protect_last_n=20`'yi sert taban yaparsak, bir sürü hacimli tool çıktısını budanamaz halde 'dondurur' — eski 'hiçbir şey sıkışmıyor' vakası geri gelir."* Yani 20 çok koruyucu; 8 hem yakın bağlamı korur hem büyük çıktıların budanmasına izin verir.

İnce matematik uyarısı (koddan)

Bütçe-yürüyüşünü "korunan sayı"ya çevirip taban **count-space'te** uygulanır, sonra tekrar sınıra çevrilir:

```
budget_protect_count = len(result) - boundary
protected_count = max(budget_protect_count, min_protect) # count-space'te max doğru okunur
prune_boundary = len(result) - protected_count
```

Neden? Yorum: *"index-space'te `max` yönü ters çevirir (küçük index = DAHA çok korunan), o yüzden cömert bir bütçe sessizce `min_protect`'e kırılırdı."* — pozisyon/sayı çevriminde işaret hatası tuzağı.

3. Pass 1 — Dedup (byte-identik tool sonuçlarını tekilleştir)

Sorun: Model aynı dosyayı 5 kez okuyabilir (`read_file a.py` × 5). Beş özdeş 40K blok = 200K boşa token.

Çözüm: Sondan geriye yürü, her tool sonucunun içeriğini **hash'le**. Aynı hash tekrar görülürse: **en yeni tam kopyayı tut**, eskileri bir **geri-referansla** değiştir.

```
content_hashes: dict = {} # hash -> (index, tool_call_id)
for i in range(len(result)-1, -1, -1):
    msg = result[i]
    if msg.get("role") != "tool": continue
    content = msg.get("content") or ""
    if isinstance(content, list): continue # multimodal -> atla
    if not isinstance(content, str): continue # dict-envelope -> hash'lenemez, atla
    if len(content) < 200: continue # dedup floor (küçükler değişmez)
    # ... hash -> görülmüşse geri-referansla değiştir
```

Kritik özellikler:

- **Tail-agnostik:** Dedup **korunan kuyruk dahil HER YERDE** çalışır. Neden güvenli? Çünkü **kayıpsız (lossless)** — en yeni tam kopya korunur, sadece *özdeş* eskiler referansa iner. Hiçbir benzersiz içerik kaybolmaz.
- **Dedup floor = 200 karakter:** 200'ün altını dedup etme (kazanç yok, gürültü).
- **Multimodal atlama:** Görsel içeren tool sonuçları (list veya `{_multimodal:True}` zarfı) metin olarak hash'lenemez → dokunulmaz.

Örnek:

```
ÖNCE:
[tool call_3] read a.py -> <40K içerik>
[tool call_8] read a.py -> <aynı 40K içerik>   ← özdeş
SONRA:
[tool call_3] read a.py -> [Duplicate of call_8's result - see below]
[tool call_8] read a.py -> <40K içerik>       ← en yeni tam kopya korunur
```

4. Pass 2 — Informative summary (büyük sonucu bilgi taşıyan satıra indir)

Sınırdan **önceki** her tool sonucu, `_demote_tool_result_at(idx)` ile budanır. Ama **her şey budanmaz** — önce elemeler:

```
def _demote_tool_result_at(idx):
    content = result[idx].get("content") or ""
    if content.startswith("(") and " chars)" in content and len(content) < 400:
        return False # zaten budanmış (kendi çıktımızı tekrar budama)
    if content.startswith("[screenshot removed]"):
        return False # zaten kaldırılmış
    if len(content) <= min_prune_chars: # varsayılan 200
        return False # küçük - budamaya değmez
    # skill_view koruması (#32106) - aşağıda
    tool_name, tool_args = call_id_to_tool.get(call_id, ("unknown", ""))
    summary = _summarize_tool_result(tool_name, tool_args, content)
    result[idx] = {**msg, "content": summary} # İÇERİĞİ değiştir, mesajı SİLME
```

```
pruned += 1
```

Dört eleme (neyi budaMA):

1. **Zaten budanmış** — içerik [...chars] şeklinde ve <400 → kendi çıktımızı tekrar özetlersek her turn bozulur (yakınsamaz).
2. **Zaten kaldırılmış screenshot** — [screenshot removed....]
3. **Küçük** — ≤ min_prune_chars (200). Bir özet, 200 karakterden kısa bir şeyi büyütebilir → asla küçültme.
4. **Korunan skill (#32106)** — aşağıda.

İçerik yerine `_summarize_tool_result` — informatif tek satır. Tüm tool dalları:

Tool	Girdi → Üretilen satır
terminal	[terminal] ran `npm test` -> exit 0, 47 lines output (regex ile exit_code çekilir, komut 80 kar. kırpılır)
read_file	[read_file] read config.py from line 1 (3,400 chars)
write_file	[write_file] wrote auth.py (N lines) (content'teki \n sayısı)
search_files	[search_files] content search for 'compress' in agent/ -> 12 matches
web_search	[web_search] query='langgraph subagent' (8,400 chars result)
web_extract	[web_extract] https://x.com (+2 more) (12,000 chars) (dict→url unwrap, aşağıda)
delegate_task	[delegate_task] 'auth'u denetle' (3,200 chars result) (goal 60 kar.)
execute_code	[execute_code] `print(df.head())...` (14 lines output) (kod 60 kar.)
browser_*	[browser_navigate] https://... (6,000 chars)
skill_view	[skill_view] name=X (5,200 chars) <pruned-marker> (#32106, aşağıda)
bilinmeyen	[<tool>] (N chars result) (backstop)

Neden jenerik placeholder değil? [Old tool output cleared] sıfır bilgi taşır → model "npm test geçti mi?" diye tekrar çalıştırır. -> exit 0, 47 lines kararı verir, tekrar çağrı gerekmez.

Asla çökmez: `_summarize_tool_result` her zaman try/except'le sarılı:

```
try: return _summarize_tool_result_unguarded(...)
except Exception:
    return f"[{tool_name}] ({len(content):,} chars result)" # backstop
```

Neden? Argümanlar *kalıcı history'den* gelir; model bazen bozuk (bool/int/None) üretir. Bozuk bir tarihsel çağrı budamayı çökertirse, budama **aynı history'de retry** eder → sonsuz **crash-loop**.

Gerçek kenar-vaka (web_extract dict→str): web_search sonucu {"url":...} dict'idir; model bunu doğrudan web_extract'a forward eder. Kod `urls[0]` dict ise unwrap eder (`.get("url")` or `.get("href")`) — yoksa aşağıdaki += işlemi **dict + str TypeError** atıp tüm ön-budamayı çökertir.

Ghost-skill savunması (#32106): skill_view çıktısı `_SKILL_VIEW_PRUNE_MIN_CHARS = 5000`'i aşarsa budanır — ama düz metadata özeti **tehlikeli**:

```
if content_len > 5000:
    return f"[skill_view] name={name} ({content_len:,} chars) " + _skill_pruned_marker(name)
```

Neden? Sadece "skill_view name=X (5000 chars)" yazarsan model skill'in **hâlâ yüklü** olduğunu sanır

("hayalet skill") ve talimatlarına göre davranır — ama içerik gitti! Kanonik marker modele: (1) *talimatlar YOK artık*, (2) *geri almak için `skill_view`'ı tekrar çağır*. Ayrıca **aktif skill koruması**: `skill_view` sonucu, o skill "az önce yüklendi/aktif referanslı" ise (`protected_skills`) **budanmaz** (verbatim kalır) — Pass-4 basınç demotion'ı hariç.

5. Pass 3 — Tool_call argümanlarını kısalt (assistant tarafı)

Sadece tool *sonuçları* değil, tool *çağrılarının argümanları* da dev olabilir. Klasik örnek: `write_file` 50KB içerikle çağrılır — bu argüman `assistant.tool_calls[].function.arguments` içinde durur ve Pass 2'den (o tool sonucunu değil çağrıyı budar) etkilenmez.

```
def _truncate_tool_call_args_at(idx):
    for tc in msg["tool_calls"]:
        args = tc["function"]["arguments"]
        if len(args) > 500:
            new_args = _truncate_tool_call_args_json(args) # JSON İÇİNDE kısalt
```

`_truncate_tool_call_args_json` argümanları **ayrıştırılmış JSON yapısı içinde** kısaltır (500+ karakterlik alanları `[:200]+"...[truncated]"`) — **geçerli JSON kalır**.

Neden geçerli JSON şart? Koddaki yorum: *"aksi halde downstream sağlayıcılar, bozuk çağrı pencereden düşene kadar her turn 400 döner."* Yani ham string kesme = bozuk JSON = kalıcı 400.

Kapsam: Pass 3 yalnız sınırdan önceki (`prune_boundary`) assistant mesajlarında çalışır — korunan kuyruktaki çağrılara dokunmaz.

6. Pass 4 — Basınç demotion'ı (#61932)

Korunan bölgenin *kendisi* hâlâ yumuşak kuyruk bütçesini aşıyorsa (`protect_tail_tokens * 1.5`), bir **basınç pass'i** korunan bölgedeki büyük *tamamlanmış* tool/dosya çıktılarını bile demote eder — ama `_PRESSURE_KEEP_RECENT_MESSAGES = 3` son mesaj hep verbatim kalır (aktif kullanıcı isteği + en yeni tool çifti okunur olsun).

Örnek: Kuyruкта tek bir 50K `read_file` var, bütçe 20K. Normalde korunurdu; ama korunan bölge bütçe×1.5'i aştığı için basınç pass'i o çıktıyı informatif özete indirir — son 3 mesaj hariç.

7. Proactive yol — `prune_tool_results_only` (maliyet motoru)

Aynı 3 pass'i **LLM özet fazı olmadan**, full-compression eşiğinden **bağımsız** çalıştırır.

Neden ayrı? Büyük-pencereli modellerde `should_compress()` (≈%50) nadiren tetiklenir; o zamana kadar eski tool çıktıları history'de **her turn tekrar tekrar gönderilir**. Bu yol onları erken geri kazanır — **kalite-riskli LLM özeti olmadan**.

Tail koruması burada COUNT ile (`protect_last_n`), token bütçesiyle DEĞİL. Neden? Token bütçesi %50 eşikten türetilir (1M pencerede ≈100K) → tüm oturumu korur, hiçbir şey budanmaz.

Üç kapı (hepsi fayda-freni):

- `proactive_prune_tokens <= 0` → kapalı, INPUT'u aynen döndür.
- `current_tokens < proactive_prune_tokens` → henüz erken.

3. `before < _proactive_prune_rearm_tokens` → yeniden-kurulum bekliyor.

Pass başına eşik: Sadece Pass 2'nin tabanı `proactive_prune_min_result_chars = 8000`'e yükseltilir (proaktif yolda sadece gerçekten büyük sonuçlar hedeflenir). Pass 1 (dedup) ve Pass 3 kendi sabit tabanlarını korur. Dedup tail-agnostik (kayıpsız).

PROMPT-CACHE SÖZLEŞMESİ (en kritik detay)

Budama, sağlayıcının **zaten gördüğü** mesaj gövdelerini yeniden yazar → en erken yeniden-yazılan mesajdan itibaren **cache'lenmiş prefix'i geçersiz kılar** (tıpkı bir compression boundary gibi).

Bu yüzden budama **ancak** `proactive_prune_min_reclaim_tokens = 4096` token geri kazanıyorsa **commit eder**; sonra history yeniden bir tetik-boyutu "runway" kadar büyüyene dek **disarm** olur (kendini kapatır). Bu iki kapının altında:

```
return messages, 0 # INPUT nesnesi aynen döner (no-op)
```

Çağırın taraf sözleşmesi: `result is not input` ise bookkeeping yapılır (yani gerçekten değişti mi diye kimlik karşılaştırması).

Neden bu kadar dikkatli? Her budama cache'i bozar. Küçük bir budama için cache'i bozmak, kazandığından fazlasına mal olur. Bu yüzden "yeterince kazandırmıyorsan, dokunma" — cache-farkındalıklı fayda-freni.

Kapasite kapısı: Pahalı 3-pass taramasından **önce**, session store atomik kalıcılaştırma yapabiliyor mu bakılır (`archive_and_compact`). Yapamıyorsa (duck-typed/plugin store) budama kalıcı olamaz → her seferinde no-op olacağına, taramayı hiç yapma.

8. Uçtan uca örnek — tam bir trace, önce → sonra

ÖNCE:

```
[assistant] read_file("a.py")
[tool call_1] <40K a.py içeriği> ← eski
[assistant] search_files("login","auth/")
[tool call_2] 47 eşleşme, 300 satır (6K) ← eski
[assistant] read_file("a.py") (tekrar!)
[tool call_3] <aynı 40K a.py içeriği> ← özdeş
[assistant] write_file("a.py", <50K yeni içerik>) ← dev argüman
[tool call_4] "wrote a.py"
[assistant] terminal("pytest")
[tool call_5] <2000 satır test çıktısı> (15K) ← YAKIN (korunur)
[assistant] Testler geçti. ← YAKIN (korunur)
```

SONRA (Pass 1→2→3, son 8/tail korunur):

```
[assistant] read_file("a.py")
[tool call_1] [read_file] read a.py from line 1 (40,000 chars) ← Pass2 informatif
[assistant] search_files("login","auth/")
[tool call_2] [search_files] content search for 'login' in auth/ -> 47 matches ← Pass2
[assistant] read_file("a.py")
[tool call_3] [Duplicate of call_1's result] ← Pass1 dedup
[assistant] write_file("a.py", {"path":"a.py","content":"<...200 kar...>...[truncated]"}) ← Pass3
arg kısaltma
[tool call_4] "wrote a.py"
[assistant] terminal("pytest")
```

[tool call_5] <2000 satır test çıktısı> (15K) ← korundu (yakın)
[assistant] Testler geçti. ← korundu

Sonuç: ~110K → ~16K. Her `tool_call_id` korundu (0 tane 400 riski). Hiçbir benzersiz bilgi kaybolmadı — a.py'nin içeriği `call_1`'de özetlendi ama "40K okundu" bilinir; `call_3` zaten kopyaydı; write argümanı kısaldı ama "a.py'ye yazıldı" bilinir; test çıktısı ve son mesaj korundu.

9. Guard/sabit tablosu (tek bakışta)

Sabit / kapı	Değer	Rolü
<code>_MAX_TAIL_MESSAGE_FLOOR</code>	8	Korunan minimum son mesaj (20 dondururdu)
<code>_PRESSURE_KEEP_RECENT_MESSAGES</code>	3	Basınçta bile verbatim kalan son mesaj (#61932)
dedup floor	200 kar.	Bunun altını dedup etme
<code>min_prune_chars</code>	200 (varsayılan)	Pass 2: bunun altını özetleme (büyütür)
<code>_SKILL_VIEW_PRUNE_MIN_CHARS</code>	5000	<code>skill_view</code> bunun üstündeyse buda (+ghost-marker)
tool_call arg eşiği	500 kar.	Pass 3: bunun üstünü kısalt
arg kısaltma head	200 kar.	JSON içi alanı <code>[:200]+truncated</code>
<code>proactive_prune_min_result_chars</code>	8000	Proaktif Pass 2 tabanı
<code>proactive_prune_min_reclaim_tokens</code>	4096	Bu kadar kazanmıyorsa commit etme (cache)
<code>_PRUNED_TOOL_PLACEHOLDER</code>	metin	Son çare jenerik ("cleared to save context")

10. POC eşlemesi — bizim tool-trace compaction'ımız

Bizim POC (poc/)	Hermes tool-trace karşılığı
<code>_render_messages</code> — tool gövdesini fate ile yeniden yaz, <code>tool_call_id</code> koru	Pass 2 — <code>result[idx]={**msg, "content":summary}</code> , id korunur
fate=ÖZET (informatif not)	<code>_summarize_tool_result</code> (tool-tipine özel satır)
fate=DEDUP	Pass 1 — byte-identik dedup, en yeni tam kopya
Fayda-freni <code>est (note) <raw</code>	<code>min_prune_chars/200-floor</code> + <code>proactive_prune_min_reclaim_tokens</code>
<code>RECENT=2</code> pozisyon koruması	<code>_MAX_TAIL_MESSAGE_FLOOR=8</code> + token-tail + <code>_PRESSURE_KEEP_RECENT=3</code>
<code>tool_call_id</code> bütünlüğü (400 riski)	mesaj silinmez, içerik değişir; Pass 3 JSON geçerli kalır
— (bizde yok)	Pass 3 argüman kısaltma · ghost-skill marker (#32106) · prompt-cache commit sözleşmesi · basınç pass'i (#61932)

Ders: Bizim POC'un tam kalbi (fate ile tool gövdesi yeniden yazma + informatif özet + dedup + id koruma + fayda-freni + son-N) Hermes'te **birebir** var. Eksiğimiz üç şey: (1) tool-çağrı-argümanı budama, (2) prompt-cache'i bozmadan commit disiplini, (3) ghost-skill gibi domain-özel işaretler.

Kaynaklar

- [../harnesses/hermes-agent/agent/context_compressor.py](https://github.com/0x00sec/0x00sec/blob/master/agent/context_compressor.py):
 - `_prune_old_tool_results` (3 pass + boundary) · `prune_tool_results_only` (proaktif + cache sözleşmesi) · `_summarize_tool_result[_unguarded]` (tüm tool dalları) · `_truncate_tool_call_args_json` (Pass 3) · sabitler (`_MAX_TAIL_MESSAGE_FLOOR`, `_PRESSURE_KEEP_RECENT_MESSAGES`, `_SKILL_VIEW_PRUNE_MIN_CHARS`, `_PRUNED_TOOL_PLACEHOLDER`).
- Issue referansları (koddan): #61932 (basınç keep-recent), #32106 (ghost-skill).
- İlgili: [hermes-agent-harness.md](#) §6 · [poc/](#) tool-trace compaction.

OpenClaw — Tool-Trace Compaction: Baştan Sona Tam Rehber (Teknik)

Not: Bu, OpenClaw tool-trace belgesinin **teknik / mühendislik** sürümüdür. Aynı içeriğin akılda-kalıcı "dadı" anlatımı için bkz. [openclaw-tool-trace-compaction.md](#).

Amaç: OpenClaw'ın tool-trace / context compaction'ını **tek bir örnek trace'i tüm adımlardan geçirerek**, her adımda somut veriyle (öncesi→sonrası) anlatmak. Kaynak: [../harnesses/openclaw/src/agents/](#) — `compaction-planning.ts`, `compaction-planning.worker.ts`, `session-transcript-repair.ts`, `agent-compaction-constants.ts`.

0. Önce en kritik fark: OpenClaw ≠ Hermes felsefesi

Hermes tool-trace'i **deterministik** budar (LLM'siz: dedup + informatif tek-satır + arg kırpma). **OpenClaw bunu yapmaz.** OpenClaw'ın yaklaşımı:

Sanitize et → **tool-çiftlerini bozmadan chunk'la** → **chunk'ları bir LLM'e özetlet.**

Yani OpenClaw'ın tool-trace mantığı esasen bir **LLM-chunk-özetleme** hattıdır. "Tool-trace"e özgü kısımlar şunlar:

- Güvenlik şeritlemesi** — tool sonucunun hassas `details`'i özetleyici modele asla girmez.
- Tool-çifti bütünlüğü** — bir `call/result` çifti asla farklı chunk'lara bölünmez.
- Oversized tool sonucu** → **not** — özet isteğine sığmayan dev mesaj, içerik yerine bir nota iner.
- Transcript onarımı** — yetim (eşi kayıp) tool sonuçları sentetik sonuçla onarılır.

Ve tüm **planlama ayrı bir worker-thread'de** yapılır (ana döngü bloklanmaz).

1. Sözlük — her terim

Terim	Anlamı
context window	Modele gönderilen tüm mesaj yığını için token sınırı.
prompt budget	Bu pencerede <i>prompt'a</i> (mesajlara) ayrılan pay; reserve token'lar düşüldükten sonra kalan.
sanitize	Özetlemenden önce hassas/model-görünmez içeriği ayıklama (<code>stripToolResultDetails</code> + <code>runtime-context</code>).
toolResult.details	Bir tool sonucunun ham/iç detay alanı (komut çıktısı iç bilgisi). Modele gösterilmez.
runtime-context	Yalnızca çalışma-zamanına ait, model-görünmez mesajlar (0 token sayılır).
group (atomik grup)	Bir tool <code>call</code> + <code>result</code> batch'inin bölünemez birimi.
pendingToolCallIds	Henüz sonucu gelmemiş tool çağrı id'leri kümesi; grup ancak bu boşalınca kapanır.
chunk	Bir özet isteğine gidecek mesaj paketi (birden çok atomik grup içerebilir).
chunk ratio	Bir chunk'ın pencerenin ne kadarını hedefleyeceği (BASE 0.4).

adaptive ratio	Mesajlar büyükse chunk oranını küçültme (0.4 → 0.15).
oversized	Tek başına özet isteğine sığmayacak kadar büyük mesaj.
oversizedNote	Oversized mesajın içeriği yerine konan kısa metin not.
stage split	Özetlemenin tek chunk mı (single) çok chunk mı (split) yapılacağı kararı.
transcript repair	Compaction sonrası call/result eşleşmesini geçerli tutma (yetim sonuç → sentetik).
worker thread	Ana döngüden ayrı iş-parçacığı; planlama burada koşar.
SAFETY_MARGIN	Token tahmini yanılma payı (1.2 = %20 pay).

2. Sabitler (agent-compaction-constants.ts + compaction-planning.ts)

```
// Tetik / bütçe
MIN_PROMPT_BUDGET_TOKENS = 8_000    // prompt'a ayrılacak mutlak taban
MIN_PROMPT_BUDGET_RATIO = 0.5        // pencerenin en az yarısı prompt'a kalmalı

// Chunk'lama
BASE_CHUNK_RATIO = 0.4                // bir chunk pencerenin %40'ını hedefler
MIN_CHUNK_RATIO = 0.15                // adaptif küçültmenin alt sınırı
SAFETY_MARGIN = 1.2                   // token tahmini yanılma payı
DEFAULT_PARTS = 2                     // stage split varsayılan parça sayısı
SUMMARIZATION_OVERHEAD_TOKENS = 4096 // özet prompt+system+önceki özet+sarmalayıcı payı

// Projeksiyon (compaction-planning-projection.ts)
PLANNING_MAX_CHARS = 256 * 1024      // planlama projeksiyonu toplam bütçe
TEXT_TRUNCATE_THRESHOLD_CHARS = 32_768 // bunun altı tam tutulur
TEXT_SAMPLE_CHARS = 8_192            // büyükse sadece 8KB örnek
```

3. Mimari — hattın tamamı

```
%% Diyagram (mermaid kaynağı):
flowchart TB
    T["0. Tetik: prompt budget < %50 (veya < 8K)"] --> S["1. Sanitize (SECURITY)"]
    S --> E["2. Token tahmini (per-mesaj, sanitized)"]
    E --> P["3. Planlama projeksiyonu"]
    P --> R["4. Adaptif chunk oranı"]
    R --> G["5. Tool-çiftine göre gruptlama"]
    G --> C["6. Chunk'lama (çift-korumalı)"]
    C --> O["7. Oversized fallback (dev → not)"]
    O --> SS["8. Stage split (single/split)"]
    SS --> W["9. Worker-thread'de planla"]
    W --> L["10. Chunk'ları LLM'e özetlet"]
    L --> RE["11. Transcript onarımı"]
    RE --> A["12. Uygula + usage snapshot geçersizle"]
```

4. Adım adım — TEK örnek trace tüm hattın geçiyor

Aşağıdaki adımların **hepsi aynı örnek trace üzerinde** ilerler; her adımda verinin o anki hâlini (öncesi→sonrası) gösteririm. Başlangıç trace'imiz (context window = **200.000 token**):

```
#0 [system]      "Sen OpenClaw'sın. auth modülünü refactor et."      ~2.000 tok
```

```
#1 [user]          "auth modülünü refactor et, login akışını sadeleştir"      ~30 tok
#2 [assistant]    tool_calls: read_file(auth.py)=c1, search_files(login)=c2
#3 [toolResult c1] auth.py'nin tamamı + details:{raw_stdout, cwd, env}      ~40.000 tok
#4 [toolResult c2] "47 eşleşme..." + details:{grep_flags}                  ~8.000 tok
#5 [assistant]    tool_calls: read_file(big_generated.py)=c3
#6 [toolResult c3] 200KB üretilmiş dosya + details                        ~110.000 tok
#7 [assistant]    "Refactor planı: login()'i böl, token'ı httpOnly yap..." ~40 tok
#8 [runtime]      <runtime-context: iç durum, model-görünmez>              (0 tok sayılır)
```

Adım 0 — Tetik (bütçe-tabanlı)

Ne: Compaction, prompt'a yeterli yer kalmayınca tetiklenir. **Kural:** Pencerenin en az yarısı prompt'a kalamıyorsa (`MIN_PROMPT_BUDGET_RATIO = 0.5`) veya taban `MIN_PROMPT_BUDGET_TOKENS = 8_000` aşılıyorsa.

Örnekte:

```
Toplam trace ≈ 2.000+30+40.000+8.000+110.000+40 ≈ 160.070 tok
Boş kalan = 200.000 - 160.070 = 39.930 tok
Gerekli taban = 200.000 × 0.5 = 100.000 tok
39.930 < 100.000 → ✓ COMPACTION TETİKLENİR
```

Adım 1 — Sanitize (SECURITY, en kritik ön-adım)

Ne: Özetlemeden önce hassas + model-görünmez içerik ayıklanır. `sanitizeCompactionMessages = stripToolResultDetails ◦ stripRuntimeContextCustomMessages`

Neden: Compaction metni bir **özet modeline** gider. Tool sonucunun `details`'ı (ham stdout, env, cwd) o modele giderse = hassas veri sızması.

Örnekte — #3 mesajı:

```
// ÖNCE
{ "role":"toolResult", "id":"c1",
  "content":"<auth.py 40K>",
  "details":{"raw_stdout":"...", "cwd":"/home/user/secret", "env":{"API_KEY":"..."} } }
// SONRA (stripToolResultDetails → delete .details)
{ "role":"toolResult", "id":"c1", "content":"<auth.py 40K>" }
```

Ve **#8 runtime mesajı** tamamen çıkarılır (model-görünmez). `content` gövdeleri kalır; sadece `details` ve runtime girdileri gider.

Adım 2 — Token tahmini (sanitized, per-mesaj, hizalı)

Ne: Her mesajın maliyeti ayrı ayrı, sanitized haliyle hesaplanır. `estimatePerMessageTokens` → diziyle **1:1 hizalı**; model-görünmez mesajlar **0**.

Örnekte üretilen dizi (index → token):

```
[#0]=2000  [#1]=30  [#2]=15  [#3]=40000  [#4]=8000  [#5]=8  [#6]=110000  [#7]=40  [#8]=0
```

#8 runtime olduğu için **0** (modele gitmiyor, baskı yaratmaz). Bu dizi bundan sonraki tüm adımların (chunk, oversized) girdisidir.

Adım 3 — Planlama projeksiyonu

Ne: `projectCompactionMessagesForPlanning` — transcript'in **boyut-doğru ama içerik-hafif** bir kopyasını üretir: büyük gövdeler 8KB örneğe iner, atılan karakter sayısı damgalanır.

```
if (metin ≤ 32KB && bütçeye sığıyor): return null // tam tutulur
else: sample = ilk 8KB; return { text: sample, omittedChars: toplam - 8KB }
// ağırlık = estimateTokens(sample) + ceil(omittedChars / 4); toplam bütçe 256KB
```

Neden: Planlayıcı/worker, megabaytlarca gövdeyi taşımadan doğru token baskısını görebilmeli.

Örnekte — #6 (110K):

```
content → ilk 8.192 karakter örnek
__openclawCompactionPlanningOmittedChars = 101.808
ağırlık ≈ estimateTokens(8KB) + ceil(101.808/4) ≈ 2.048 + 25.452 ≈ 27.500 tok
```

Worker #6'yı doğru "büyük" görür ama 110K'lık gövdeyi hiç taşımaz. **Güvenlik detayı:** ölçülemeyen argüman `Number.MAX_SAFE_INTEGER` sayılır — baskıyı asla eksik gösterme, mutlaka oversized'a düşür.

Adım 4 — Adaptif chunk oranı

Ne: Bir chunk'ın pencerenin ne kadarını hedefleyeceği, mesaj boyutuna göre ayarlanır.

```
avgRatio = (ortalama_mesaj_token × SAFETY_MARGIN) / contextWindow
if avgRatio > 0.10:
    reduction = min(avgRatio × 2, BASE-MIN)
    ratio = max(MIN_CHUNK_RATIO, BASE_CHUNK_RATIO - reduction) // 0.4→0.15
else:
    ratio = BASE_CHUNK_RATIO // 0.4
```

Neden: Mesajlar büyükse büyük chunk özet-modelinin limitini aşar → küçük chunk daha güvenli.

Örnekte hesap:

```
ortalama = 160.070 / 8 ≈ 20.009
avgRatio = 20.009 × 1.2 / 200.000 = 0.120 → > 0.10
reduction = min(0.240, 0.25) = 0.240
ratio = max(0.15, 0.16) = 0.16
→ maxChunkTokens = 0.16 × 200.000 = 32.000 tok
```

Büyük mesajlar yüzünden hedef chunk 80K'dan (0.4) **32K'ya** indi. **İki rejim:** `avgRatio ≤ 0.10` → 0.40 (tam boy); `> 0.125` → hep 0.15 (taban); arada dar bir geçiş bandı (0.10–0.125).

Adım 5 — Tool-çiftine göre grublama (bütünlük çekirdeği)

Ne: Mesajlar, bir `call/result` batch'i **bölünmeyecek** şekilde atomik gruplara ayrılır. Grup ancak `pendingToolCallIds` boşalınca kapanır.

```
if role == "assistant":
    toolCalls = (stopReason ∈ {aborted, error}) ? [] : extractToolCallsFromAssistant(message)
    pendingToolCallIds = Set(toolCalls.map(id))
elif role == "toolResult" and pendingToolCallIds.size > 0:
```

```
resultId ? pendingToolCallIds.delete(resultId) : pendingToolCallIds.clear()
if pendingToolCallIds.size == 0:    # ← grup ANCAK burada kapanır
    groups.push(current); current = []
```

Örnekte pending kümesinin evrimi:

```
#0 system    → pending={}          → GRUP A kapanır: [#0]
#1 user      → pending={}          → GRUP B kapanır: [#1]
#2 assistant → pending={c1,c2}     → açık (2 çağrı bekliyor)
#3 result c1 → pending={c2}        → HÂLÂ açık
#4 result c2 → pending={}          → GRUP C kapanır: [#2,#3,#4] ← call+2 result bir arada
#5 assistant → pending={c3}        → açık
#6 result c3 → pending={}          → GRUP D kapanır: [#5,#6]
#7 assistant → pending={} (tool yok) → GRUP E kapanır: [#7]
```

5 grup: A[#0] B[#1] C[#2,#3,#4] D[#5,#6] E[#7]. **İnce detay:** `stopReason` aborted/error ise çağrılar [] sayılır (sonuç hiç gelmeyecek, sonsuza dek pending kalmasın).

Adım 6 — Chunk'lama (çift-korumalı)

Ne: Atomik gruplar `maxChunkTokens`'a (Adım 4: 32K) kadar chunk'lara paketlenir. Grup **asla bölünmez**.

`effectiveMax = maxChunkTokens / SAFETY_MARGIN = 32.000 / 1.2 ≈ 26.667`

Örnekte paketleme (grup token'ları: A=2000, B=30, C=48.015, D=110.008, E=40):

```
+ A(2000), + B(30)          → current=[A,B] 2.030
+ C(48.015): 2.030+48.015 > 26.667 → CHUNK 1 = [A,B]; current=[C]
+ D(110.008): > 26.667 → CHUNK 2 = [C]; current=[D]
+ E(40): > 26.667 → CHUNK 3 = [D]; current=[E]
son → CHUNK 4 = [E]
```

4 chunk: [A,B] [C] [D] [E]. C (48K) ve D (110K) tek başlarına eşiği aşıyor ama **grup bölünemediği için** tek chunk kaldılar — sonraki adımın (oversized) konusu.

Adım 7 — Oversized fallback (tek mesaj devse → not)

Ne: Bir **tek mesaj** özet isteğine sığmayacak kadar büyükse (`contextWindow × 0.5`), içeriği yerine bir **not** konur.

```
oversizedThreshold = 200.000 × 0.5 = 100.000
if tokens × SAFETY_MARGIN > oversizedThreshold:
    oversizedNotes.push(`[Large ${role} (~${tokens/1000}K tokens) omitted from summary]`)
```

Örnekte — #6 (110K):

```
#6: 110.000 × 1.2 = 132.000 > 100.000 → OVERSIZED
oversizedNote = "[Large toolResult (~110K tokens) omitted from summary]"
omitToolBatch = true → #5 (o batch'in çağrısı) da düşer
```

#3 (40K): $40.000 \times 1.2 = 48.000 < 100.000 \rightarrow$ oversized değil, normal özetlenir. **İnce detay:** Bir tool batch oversized diye atlanırsa kalan `assistant/toolResult` parçaları da düşer — ama araya sıkışmış **gerçek user mesajı** hayatta kalır.

Adım 8 — Stage split (tek mi, çok mu chunk özetlensin)

Ne: Özetlenecek mesajlar tek seferde mi (`single`) yoksa parçalara bölünerek mi (`split`).

```
minMessagesForSplit = 4
if parts<=1 or messages.length<4 or totalTokens<=maxChunkTokens: return {mode:"single"}
else: return {mode:"split", chunks: splitMessagesByTokenShare(messages, parts)}
```

Örnekte: Özetlenecek normal içerik grup C ([#2,#3,#4], 3 mesaj) — oversized D çıkarıldı.

```
messages.length = 3 (< 4) → mode: "single"
```

→ Grup C tek özet isteğinde özetlenir. (6-7 büyük mesaj kalsaydı → `split: DEFAULT_PARTS=2` ile iki eşit token-payı, yine çift bölmeden.)

Adım 9 — Worker-thread'de planla

Ne: 4-8. adımların hesabı ana döngüde değil, `compaction-planning.worker.ts` içinde **ayrı iş-parçacığında** yapılır. **Neden:** Büyük transcript'te token tahmini + gruplama + chunk'lama CPU-yoğun; ana döngüde yapılırsa ajan donar. **Örnekte:** Ana thread worker'a `{kind:"stageSplit", ...}` / `{kind:"oversizedFallback", ...}` yollar; worker planı döndürür; bu sırada ajan başka iş yapabilir.

Adım 10 — Chunk'ları LLM'e özetlet

Ne: Hazır (sanitized, çift-korumalı) chunk bir **özet modeline** verilir; `SUMMARIZATION_OVERHEAD_TOKENS = 4096` prompt/system/önceki-özet payı. **Örnekte — grup C özetlenir:**

```
Girdi: read_file(auth.py) + <auth.py 40K> + search_files(login) + "47 eşleşme"
Çıktı: "auth.py okundu (login/logout/token akışı). 'login' için 47 eşleşme;
       token httpOnly cookie'de; login() 45. satırda çift-doğrulama." (~250 tok)
```

Adım 11 — Transcript onarımı

Ne: Compaction sonrası **eşi kayıp** (yetim) call/result çiftleri onarılır. `repairToolUseResultPairing` eksik sonuç için **sentetik hata sonucu** ekler:

"[openclaw] missing tool result in session history; inserted synthetic error result for transcript repair."
`sanitizeToolCallInputs` bozuk çağrı girdilerini düzeltir. **Neden:** Sağlayıcı geçerli call↔result eşleşmesi ister; yoksa replay 400 döner.

Adım 12 — Uygula + usage snapshot geçersizle

Ne: Yeni transcript oturuma yazılır. `stripStaleAssistantUsageBeforeLatestCompaction` — compaction öncesi assistant **usage snapshot'larını sıfırlar** (`makeZeroUsageSnapshot`), çünkü o token sayıları artık geçersiz.

Örnekte SONRA (final transcript):

```
#0 [system]      "Sen OpenClaw'sın..."      ~2.000 tok
```

#1 [user]	"auth modülünü refactor et..."	~30 tok	
#2 [summary]	"auth.py okundu... 47 eşleşme... login() 45. satır..."	~250 tok	← grup C özeti
#3 [note]	"[Large toolResult (~110K tokens) omitted from summary]"	~15 tok	← grup D
#4 [assistant]	"Refactor planı: login()'i böl, token httpOnly..."	~40 tok	← TAIL korundu

160.070 → ~2.335 token (%98 kazanç)
tüm call/result eşleşmeleri geçerli · user sözü korundu · hassas details sızmadı

5. Tüm hattın özeti (tek bakış)

Adım	Girdi	İşlem	Örnekteki sonuç
0 Tetik	160K/200K	boş < %50?	39.9K < 100K → tetik
1 Sanitize	ham mesajlar	.details + runtime sil	#3 details silindi, #8 çıktı
2 Estimate	sanitized	per-mesaj token	[2000, 30, 15, 40000, 8000, 8, 110000, 40, 0]
3 Project	mesajlar	8KB örnek + omittedChars	#6 → 8KB + "atılan 101.808"
4 Adaptif	ort. mesaj	oran 0.4 → ?	avgRatio 0.12 → ratio 0.16 → 32K
5 Grupla	mesajlar	çifti bozma	5 grup: A B C[#2-4] D[#5-6] E
6 Chunk	gruplar	≤32K paketle	4 chunk: [A,B][C][D][E]
7 Oversized	chunk'lar	dev → not	#6 → "~110K omitted", #5 düşer
8 Stage split	kalan	single/split	3 mesaj < 4 → single
9 Worker	plan isteği	ayrı thread	ana döngü bloklanmaz
10 LLM özet	grup C	özetlet	"auth.py... 47 eşleşme..."
11 Onar	sonuç	yetim → sentetik	eşleşme geçerli
12 Uygula	yeni transcript	yaz + usage sıfırla	160K → 2.3K (%98)

6. Hermes ile fark (iki "asistan" ajanı)

Eksen	Hermes	OpenClaw
Ana yöntem	deterministik 3-pass (dedup + informatif satır + arg kırpma)	LLM-chunk özetleme (grupla → chunk → özetlet)
Tool sonucu	tip-farkında tek satıra iner (LLM'siz)	detay şeritlenir, chunk'lanır, modele özetletilir
Çift bütünlüğü	mesaj silinmez, id korunur	grup bölme yasağı (pending boşalınca kapat)
Dev tek mesaj	informatif özet	oversizedNote ("~NK omitted")
Güvenlik	özet sınırı redaksiyonu (#14665)	toolResult.details şeritleme (özetleyiciye sızmasın)
Nerede koşar	ana + kilitli	ayrı worker-thread
Onarım	—	sentetik sonuç (yetim call/result)
Maliyet	ucuz (çoğu LLM'siz)	LLM özeti gerekir (pahalı ama esnek)

Tek cümle: Hermes "tool sonucunu deterministik kurallarla küçült"; OpenClaw "tool-çiftlerini güvenle grupta, modele özetlet".

7. POC eşlemesi — bizim iş nerede

Bizim POC (poc/)	OpenClaw karşılığı
tool_call_id bütünlüğü (API 400)	grup bölme yasağı (pendingToolCallIds)
Fayda-freni	oversized eşiği (contextWindow×0.5)
Referansa indirme	oversizedNote ("~NK omitted")
fate işaretleme	— (OpenClaw yeniden-yazmaz, chunk'lar)
— (bizde yok)	worker-thread planlama · güvenlik-şeritleme · transcript onarımı · adaptif chunk oranı

Not: OpenClaw, bizim POC'un "tek tool-mesajı yeniden yazma" modelinden farklı bir okuldadır — o **chunk-özetleme** okulu (Codex-history ve Claude Code'a yakın). Bizim POC ise Hermes/OpenCode okuluna (deterministik per-tool) daha yakın. En net ayrım: *tool sonucunu yerinde mi küçültüyorsun (deterministik), yoksa bir modele mi özetletiyorsun (chunk)?*

Kaynaklar

- [../harnesses/openclaw/src/agents/compaction-planning.ts](#) — groupCompactionMessages · chunkCompactionMessageGroups · computeAdaptiveChunkRatio · buildSummaryChunks · buildOversizedFallbackPlan · buildStageSplitPlan · sanitizeCompactionMessages · estimatePerMessageTokens
- compaction-planning-projection.ts — projectCompactionPlanningMessages (8KB örnek + omittedChars, 256KB bütçe)
- compaction-planning.worker.ts — worker-thread giriş noktası
- session-transcript-repair.ts — stripToolResultDetails · repairToolUseResultPairing · sanitizeToolCallInputs
- agent-compaction-constants.ts — MIN_PROMPT_BUDGET_TOKENS/RATIO
- embedded-agent-subscribe.handlers.compaction.ts — handleCompactionStart/End
- compaction-usage.ts — stripStaleAssistantUsageBeforeLatestCompaction
- Akılda-kalıcı "dadı" sürümü: [openclaw-tool-trace-compaction.md](#)
- Eş belgeler: [hermes-tool-trace-compaction.md](#) · [opencode-tool-trace-compaction.md](#) · [codex-tool-trace-compaction.md](#)

OpenCode — Tool-Trace Compaction: Baştan Sona Tam Rehber

Amaç: OpenCode'un tool-trace / context compaction'ını **tek bir örnek trace'i tüm adımlardan geçirerek**, her adımda somut veriyle (öncesi→sonrası) anlatmak — [OpenClaw](#) ve [Hermes](#) belgeleriyle aynı derinlikte. Kaynak: [./harnesses/opencode/packages/opencode/src/](#) — `tool/truncate.ts`, `session/compaction.ts`, `session/overflow.ts`.

0. Kimlik / felsefe — iki bağımsız katman (POC'umuza en yakın)

OpenCode iki ayrı mekanizma kullanır:

- Katman A — canlı tool-output spill (üretim anında):** Bir tool çıktısı üretilir üretilmez, çok büyükse **diske yazılır**, context'e sadece önizleme + "kayıtlı dosyaya bak" ipucu girer.
- Katman B — deterministik prune + overflow LLM özeti (eşikte):** Context dolunca, eski tool çıktıları önce **deterministik işaretlenip** kısaltılır (LLM'siz); yetmezse turn'ler **LLM'e özetletilir**.

Sabitleri bizim POC'a şaşırtıcı yakın: `DEFAULT_TAIL_TURNS=2` (=bizim `RECENT=2`), `PRUNE_PROTECTED_TOOLS=["skill"]` (=korunan tool), `compacted` timestamp (=bizim `fate`).

1. Sözlük

Terim	Anlamı
part	Bir mesajın içindeki birim (metin, tool çağrısı, tool sonucu). OpenCode mesajları <code>parts[]</code> taşır.
tool part	<code>type: "tool"</code> bir part; <code>state.status</code> , <code>state.output</code> , <code>state.time.compacted</code> alanları var.
spill (dökme)	Büyük tool çıktısını diske yazıp context'te referans/önizleme bırakma.
truncation-dir	Dökülen tam çıktının saklandığı disk klasörü (7 gün).
prune	Deterministik tool-output işaretleme (<code>compacted</code> damgası) + <code>serialize</code> 'da kısaltma.
compacted (damga)	<code>part.state.time.compacted</code> — bu tool çıktısının sıkıştırıldığını işaretleyen zaman damgası (bizim <code>fate</code>).
turn	Bir <code>user</code> mesajıyla başlayan konuşma bloğu.
usable	Pencerede prompt'a ayrılan kullanılabilir token (reserve düşülmüş).
overflow	Kullanılan token <code>usable</code> 'ı aşınca oluşan taşma → LLM özeti tetiklenir.
preserve recent budget	LLM özetinde korunan son-pencere token bütçesi.

2. Sabitler

```
// Katman A — tool/truncate.ts
```

```
MAX_LINES = 2000          // tool çıktısı bu satırı aşarsa dök
MAX_BYTES = 50 * 1024      // ya da bu baytı (50KB)
RETENTION = 7 gün         // dökülen dosya saklama süresi
direction = "head" | "tail" // önizlemede baş mı son mu tutulsun

// Katman B - session/compaction.ts
PRUNE_MINIMUM = 20_000     // budama en az bu kadar kazanmıyorsa commit etme (fayda-freni)
PRUNE_PROTECT = 40_000     // en yeni bu kadar tool çıktısı korunur
TOOL_OUTPUT_MAX_CHARS = 2_000 // compacted çıktı serialize'da bu kadara iner
PRUNE_PROTECTED_TOOLS = ["skill"] // asla budanmayan tool
DEFAULT_TAIL_TURNS = 2     // son 2 turn dokunulmaz
MIN/MAX_PRESERVE_RECENT_TOKENS = 2_000 / 8_000

// overflow.ts
COMPACTION_BUFFER = 20_000 // reserve token (çıktı için ayrılan)
```

3. Mimari

```
%% Diyagram (mermaid kaynağı):
flowchart TB
    subgraph A["Katman A - canlı (her tool çıktısında)"]
        A1["tool çıktısı üret"] --> A2["> 2000 satır / 50KB?"]
        A2 -->|evet| A3["diske dök + önizleme+ipucu döndür"]
        A2 -->|hayır| A4["olduğu gibi bırak"]
    end
    subgraph B["Katman B - eşikte"]
        B1["isOverflow: kullanılan ≥ usable?"] -->|prune ilk| B2["deterministik prune (compacted damga)"]
        B2 --> B3["serialize: compacted → 2000 kar."]
        B1 -->|hâlâ taşma| B4["turn'leri LLM'e özetlet"]
    end
    A --> B
```

4. Tek örnek trace, tüm adımlardan

Başlangıç (model context = 200.000 token):

```
#0 [system]      "Sen OpenCode'sun..." ~2.000 tok
#1 [user]        "auth modülünü refactor et" ~30 tok
#2 [assistant]   part: tool bash("pytest auth/")
#3 [assistant]   part: tool(bash) state.output = 2.500 satırlık test çıktısı (üretim anı)
#4 [user]        "şimdi login()'i sadeleştir"
#5 [assistant]   part: tool read_file(auth/login.py)
#6 [assistant]   part: tool(read) state.output = auth.py 40K
#7 [assistant]   part: tool skill_view(github)
#8 [assistant]   part: tool(skill) state.output = 6K skill talimatı
#9 [assistant]   "login()'i böldüm, token httpOnly yaptım."
```

Adım 1 — (Katman A) Tool çıktısı üretilince canlı kontrol

Ne: #3'ün 2.500 satırlık `pytest` çıktısı üretilir üretilmez `truncate.ts` devreye girer.

```
if (satır > MAX_LINES(2000) || bayt > MAX_BYTES(50KB)): → dök
```

Örnekte: 2.500 satır > 2.000 → dökülür.

Adım 2 — (Katman A) Diske dök, önizleme+ipucu döndür

Ne: Tam metin `truncation-dir`'e yazılır; `context`'e önizleme + "kayıtlı dosyaya bak" ipucu girer.

```
// #3 state.output — ÖNCE (2.500 satır)
"..... 2500 satır test çıktısı ....."
// SONRA (context'e giren)
{ content: "<ilk ~2000 satır önizleme>\n...\n[Truncated. Full output: .opencode/truncation/
abc123.txt]",
  truncated: true, outputPath: ".opencode/truncation/abc123.txt" }
```

`direction: "tail"` verilseydi son satırlar tutulurdu (exit satırı için). Dosya 7 gün saklanır. → Bu = **spill-to-disk**: dev çıktı `context`'e hiç tam girmez.

Adım 3 — (Katman B) Overflow tetiği

Ne: Konuşma büyüdükçe her turn `isOverflow` kontrol edilir:

```
usable = model.limit.input - reserved // reserved = min(20K, maxOutput)
isOverflow = kullanılan_token ≥ usable
```

Örnekte:

```
usable = 200.000 - 20.000 = 180.000
Kullanılan (spill sonrası bile auth.py 40K + skill 6K + ...) = 185.000 ≥ 180.000 → ✓ OVERFLOW
```

Adım 4 — (Katman B) Deterministik prune: geriye yürü

Ne: `cfg.compaction.prune` açıksa, önce **LLM'siz** budama denenir. Mesajlar **sondan başa** taranır:

```
turns = 0
loop: for msgIndex = son → 0:
  if role=="user": turns++
  if turns < 2: continue // DEFAULT_TAIL_TURNS: son 2 turn dokunma
  if assistant && summary: break // önceki compaction sınırında dur
  for part in parts (sondan):
    if part.type != "tool" or status != "completed": continue
    if part.tool ∈ PRUNE_PROTECTED_TOOLS(["skill"]): continue // skill'i koru
    if part.state.time.compacted: break // zaten sıkışmış → dur
    estimate = Token.estimate(part.state.output)
    total += estimate
    if total <= PRUNE_PROTECT(40K): continue // en yeni 40K tool çıktısını KORU
    pruned += estimate
    toPrune.push(part)
```

Örnekte yürüyüş (sondan):

```
#9 assistant metin (tool yok) → atla
#8 skill_view çıktısı → PRUNE_PROTECTED → ATLA (skill korunur)
#7 (skill call)
#6 read auth.py (40K): turns<2? #4 user'dan beri 0 user geçti...
  turns sayımı: #4 user'a gelene dek turns=0 → #6,#7,#8,#9 son turn (turns<2) → DOKUNMA
#4 user → turns=1 (hâlâ <2, koru)
#3 bash çıktısı (spill sonrası önizleme, ~2K): turns=1<2 → koru
#1 user → turns=2 → artık budanabilir
```

ama #1'den öncesi sadece system → budanacak büyük tool yok

Bu örnekte son-2-turn koruması + 40K koruması çoğunu koruyor. **Daha eski, büyük bir tool çıktısı olsaydı** (turns≥2 bölgesinde, 40K'yı aşan) → `toPrune`'a girerdi.

Adım 5 — (Katman B) Fayda-freni: sadece kazanç > 20K ise commit et

Ne:

```
if (pruned > PRUNE_MINIMUM(20_000)):  
    for part in toPrune: part.state.time.compacted = Date.now(); updatePart(part)
```

Neden: Küçük bir budama (< 20K) uğraşmaya değmez. Ancak toplam prune > 20K ise işaretleme **commit** edilir. **Örnekte:** Diyelim eski bölgede 55K'lık budanabilir tool çıktısı bulundu → `55K > 20K` → hepsine `compacted` damgası basılır.

Adım 6 — (Katman B) Serialize: compacted çıktı kısılır

Ne: `compacted` damgalı tool çıktıları modele gönderilirken `TOOL_OUTPUT_MAX_CHARS = 2_000`'e iner.

```
// damgalı bir tool part - modele giden hâli  
{ tool:"read_file", state:{ output:"<ilk 2000 karakter>...[compacted]" } }
```

Damga bir **"fate" bayrağıdır** (bizim `fate` alanının muadili): part silinmez, sadece serialize'da kısılır. Böylece `tool_call` ↔ `tool_result` eşleşmesi bozulmaz.

Adım 7 — (Katman B) Hâlâ taşıyorsa: LLM özeti (processCompaction)

Ne: Prune yetmediyse turn'ler LLM'e özetletilir.

- `turns()` — mesajları turn'lere böl (her user bir turn başlatır; `compaction` part'lı user'lar atlanır).
- `completedCompactions()` — zaten özetlenmiş turn'leri izle (user'da `compaction` part'ı + assistant'ta `summary`).
- `preserveRecentBudget = min(8K, max(2K, usable×0.25))` — son pencere korunur.
- `splitTurn` — bir turn hâlâ bütçeyi aşıyorsa içinde bölme noktası bulunur.
- `summary.ts` + `compaction.txt` prompt'uyla özetlenir.

Örnekte:

```
preserveRecentBudget = min(8000, max(2000, 180000×0.25=45000)) = 8000  
→ son 8K token'lık turn'ler korunur, öncekiler özete iner:  
[summary] "auth.py okundu, pytest çalıştı (bkz .opencode/truncation/abc123.txt),  
login() sadeleştirildi, token httpOnly yapıldı."
```

Adım 8 — Medya taşması (özel durum)

Ne: Prompt sağlayıcının boyut limitini medya (ek dosya/görsel) yüzünden aşarsa: sıkıştırılır + **medya kaldırılır** ve modele açıklama enjekte edilir:

"The previous request exceeded the provider's size limit due to large media attachments... media files were removed... suggest they try again with smaller or fewer files."

5. Tüm hattın özeti (tek bakış)

Adım	Katman	İşlem	Örnekteki sonuç
1	A	çıkıtı > 2000 satır?	#3 pytest 2.500 satır → dök
2	A	diske dök + önizleme	<code>.opencode/truncation/abc123.txt</code> + preview
3	B	isOverflow?	185K ≥ 180K → taşma
4	B	deterministik prune walk	son-2-turn + 40K korundu
5	B	fayda-freni (>20K)	55K bulundu → <code>compacted</code> damgası
6	B	serialize kısaltma	damgalılar → 2.000 kar.
7	B	yetmezse LLM özeti	preserveRecent=8K, öncesi özete
8	B	medya taşması	medya kaldır + açıkla

6. Hermes / OpenClaw ile fark

Eksen	Hermes	OpenClaw	OpenCode
Tool çıktısı	informatif 1-satır (LLM'siz)	detay şeritle + chunk	diske dök (spill) + compacted damga
Ana yöntem	deterministik 3-pass	LLM-chunk özetleme	deterministik prune + gerekirse LLM
Fayda-freni	#60451	oversized eşiği	PRUNE_MINIMUM=20K
Son koruma	floor-8 + token-tail	budget %50	DEFAULT_TAIL_TURNS=2 + 40K
Korunan tool	head	—	<code>["skill"]</code>
Damga/işaret	içerik yeniden yazma	—	<code>state.time.compacted (fate)</code>

Öz: OpenCode = Hermes'in deterministik ruhu + benzersiz **spill-to-disk** + turn-tabanlı LLM özeti. Bizim POC'a en yakın akraba.

7. POC eşlemesi

Bizim POC (<code>poc/</code>)	OpenCode karşılığı
<code>RECENT=2</code>	DEFAULT_TAIL_TURNS=2 (birebir)
<code>fate</code> işaretleme	<code>part.state.time.compacted</code> damgası
korunan tool	PRUNE_PROTECTED_TOOLS=["skill"]
fayda-freni	PRUNE_MINIMUM=20_000
<code>tool_call_id</code> bütünlüğü	part silinmez, serialize'da kısalır
referansa indirme	spill-to-disk (<code>truncation-dir</code> + <code>outputPath</code>)
— (bizde yok)	canlı üretim-anı dökme · overflow LLM özeti · medya kaldırma

Kaynaklar

- [../harnesses/opencode/packages/opencode/src/tool/truncate.ts](#) — MAX_LINES · MAX_BYTES · spill · RETENTION
- session/compaction.ts — prune walk · PRUNE_MINIMUM/PROTECT · TOOL_OUTPUT_MAX_CHARS · PRUNE_PROTECTED_TOOLS · DEFAULT_TAIL_TURNS · turns/completedCompactions/preserveRecentBudget/splitTurn
- session/overflow.ts — usable · isOverflow · COMPACTION_BUFFER
- Eş belgeler: [hermes-tool-trace-compaction.md](#) · [openclaw-tool-trace-compaction.md](#) · [poc/](#)

Codex — Tool-Trace Compaction: Baştan Sona Tam Rehber

Amaç: Codex'in (openai/codex, Rust) tool-trace / context compaction'ını **tek bir örnek trace'i tüm adımlardan geçirerek**, her adımda somut veriyle anlatmak — [OpenClaw](#), [OpenCode](#), [Hermes](#) belgeleriyle aynı derinlikte. Kaynak: [../harnesses/codex/codex-rs/](#) — `utils/output-truncation/src/lib.rs`, `utils/string/src/truncate.rs`, `core/src/compact.rs`, `protocol/src/compacted_item.rs`, `prompts/templates/compact/`.

0. Kimlik / felsefe — iki katman: ortadan-kesme + "compaction bir turn'dür"

Codex iki mekanizma kullanır:

- Katman A — per-tool-output ortadan-kesme (deterministik, LLM'siz):** Büyük bir tool/exec çıktısının **başı ve sonu korunur, ORTASI atılır** — bir uyarı başlığıyla.
- Katman B — tarih compaction'ı bir MODEL TURN'ü olarak (LLM'li) + pencereli saklama:** Context dolunca, geçmiş **bir handoff özetiyle** değiştiren tam bir model turn'ü çalışır; bu özet **pencere (window)** olarak zincirlenir ve **geri sarılabilir/resume edilebilir**.

Codex'i ayıran iki şey: (1) **ortadan-kesme** (baş+son koru), (2) compaction'ın **kalıcı, pencereli, geri-sarılabilir** bir katman olması (yok etme değil).

1. Sözlük

Terim	Anlamı
TruncationPolicy	Kesme bütçesi: <code>Bytes (n)</code> veya <code>Tokens (n)</code> .
byte_budget	Politikanın bayt karşılığı (Tokens ise token→bayt yaklaşık çevrimi).
truncate_middle	Baş + son tutup ortayı atan kesme (bir işaretle).
ResponseItem	Codex'in history birimi (mesaj/tool çağırısı/çıktı).
record_items	History'ye öge yazma; kayıt anında truncation_policy uygulanır.
compaction (turn)	Geçmiş bir handoff özetiyle değiştiren tam model turn'ü.
window (pencere)	Bir compaction'ın ürettiği tarih katmanı; zincirlenir, resume edilebilir.
CompactedItem	Pencere kaydı: <code>window_number</code> , <code>first/previous_window_id</code> , <code>replacement_history</code> .
DoNotInject	Pre-turn/manuel compaction'da özet history'ye enjekte etmeme varyantı.
summary_prefix	Resume'da özetin önüne eklenen "başka bir model bunu üretti, üstüne inşa et" metni.
remote compaction	Sunucu-tarafı compaction (v1/v2).
prefix cache	Sağlayıcının baştan-aynı mesajları önbellemekmesi; baştan trim onu korur.

2. Sabitler / politikalar

```
// Katman A - utils/output-truncation + utils/string/truncate.rs
TruncationPolicy::Bytes(n)    → truncate_middle_chars(content, n)          // ortadan kes
TruncationPolicy::Tokens(n)   → truncate_middle_with_token_budget(content, n)
byte_budget()                 // Tokens ise token-bayt yaklaşık
// formatted_truncate_text → "Warning: truncated output (original token count: N)\nTotal output
lines: M\n\n..."
// InputImage / InputAudio / EncryptedContent → truncate EDİLMEZ (yalnız InputText)

// Katman B - core/src/compact.rs + prompts/templates/compact/
SUMMARIZATION_PROMPT // "CONTEXT CHECKPOINT COMPACTION. Create a handoff summary..."
summary_prefix.md     // resume'da özet önüne eklenir
CompactionTrigger     // auto (inline) | manual
CompactionImplementation::Responses
InitialContextInjection::{ BeforeLastUserMessage | DoNotInject }
```

3. Mimari

```
%% Diyagram (mermaid kaynağı):
flowchart TB
    subgraph A["Katman A - her tool/exec çıktısında (deterministik)"]
        A1["Çıktı record_items ile yazılır"] --> A2["content.len > byte_budget?"]
        A2 -->|evet| A3["truncate_middle: baş+son tut, ortayı at + Warning başlığı"]
        A2 -->|hayır| A4["olduğu gibi"]
    end
    subgraph B["Katman B - eşikte (model turn'ü)"]
        B1["auto/manual tetik"] --> B2["compaction context kur (DoNotInject?)"]
        B2 --> B3["SUMMARIZATION_PROMPT ile MODEL TURN → handoff özeti"]
        B3 --> B4["baştan trim (prefix cache koru)"]
        B4 --> B5["CompactedItem: yeni pencere, zincirle (resume edilebilir)"]
    end
    A --> B
```

4. Tek örnek trace, tüm adımlardan

Başlangıç (model context = 200.000 token):

#0 [message system]	"Sen Codex'sin..."	~2.000 tok
#1 [message user]	"auth modülünü refactor et"	~30 tok
#2 [function_call]	shell("pytest auth/") call_id=c1	
#3 [function_output c1]	2.500 satır test çıktısı	~15.000 tok
#4 [function_call]	read_file("auth/login.py") call_id=c2	
#5 [function_output c2]	auth.py + bir ekran görüntüsü (InputImage)	~40.000 tok
#6 [message user]	"login()'i sadeleştir"	
#7 [message assistant]	"Planım: ..."	~40 tok

Adım 1 — (Katman A) Çıktı record edilirken kesme kontrolü

Ne: #3/#5 history'ye record_items(&[...], truncation_policy) ile yazılır. Kayıt anında politika uygulanır:

```
if content.len() <= policy.byte_budget() { return content } // sığıyorsa dokunma
```

Örnekte: Diyelim `TruncationPolicy::Tokens(8000)` → `byte_budget ≈ 32.000` bayt. #3 (15K token ≈ 60KB) bütçeyi aşar → kesilir.

Adım 2 — (Katman A) `truncate_middle`: baş + son tut, ORTAYI at

Ne: Codex çıktının **başını ve sonunu** korur, **ortasını** atar (`truncate_middle_chars/`
`truncate_middle_with_token_budget`). **Neden:** Bir komut çıktısında en değerli kısımlar genelde **baş** (ne çalıştı, ilk hatalar) ve **son** (özet/exit). Orta genelde tekrarlı gövde.

Örnekte — #3:

```
// ÖNCE (2.500 satır)
test_login PASSED
test_logout PASSED
... (2.480 satır daha) ...
=== 47 passed, 3 failed ===
exit 1

// SONRA (truncate_middle + formatted_truncate_text)
Warning: truncated output (original token count: 15000)
Total output lines: 2500

test_login PASSED          ← baş korundu
test_logout PASSED
...[orta atlandı]...      ← orta atıldı
=== 47 passed, 3 failed === ← son korundu
exit 1
```

Model hem kesildiğini **hem ne kadar** kesildiğini bilir (Warning başlığı).

Adım 3 — (Katman A) Multimodal koruma

Ne: Yalnız `InputText` segmentleri birleştirilip kesilir; `InputImage` / `InputAudio` / `EncryptedContent` dokunulmaz. **Örnekte — #5:** `auth.py` metni kesilir (ortadan), ama ekran görüntüsü (`InputImage`) olduğu gibi kalır.

```
let text_segments = items.filter(InputText);    // sadece metin
// InputImage/InputAudio/EncryptedContent → pas
if combined.len() <= policy.byte_budget() { return items }
```

Adım 4 — (Katman B) Compaction tetiği

Ne: Context dolunca `run_inline_auto_compact_task` (otomatik) veya manuel `run_compact_task` çağrılır. `CompactionTrigger` = auto | manual. **Örnekte:** Kesmelere rağmen context eşiği aşıldı → **auto compaction** başlar.

Adım 5 — (Katman B) Compaction context'i kur

Ne: `build_compaction_initial_context` iki moda göre başlangıç bağlamı kurar:

- `BeforeLastUserMessage` — dünya-durumu (`world_state`) enjekte edilir (mid-turn).
- `DoNotInject` — özet history'ye enjekte edilmez (pre-turn/manuel).

Örnekte: auto/mid-turn → `BeforeLastUserMessage`, `world_state` ile.

Adım 6 — (Katman B) Talimatı record et (`truncation_policy` ile)

Ne: Compaction talimatı bir user girdisi olarak history'ye kaydedilir:

```
let input = vec![UserInput::Text { text: SUMMARIZATION_PROMPT, ... }];
history.record_items(&[input], truncation_policy) // kayıt anında yine kesme uygulanır
```

SUMMARIZATION_PROMPT (prompt.md):

"You are performing a CONTEXT CHECKPOINT COMPACTION. Create a handoff summary for another LLM that will resume the task. Include: current progress, key decisions, constraints, what remains, critical data/references."

Adım 7 — (Katman B) MODEL TURN: handoff özeti üret

Ne: Codex'in imzası — compaction bir **tam model turn**'üdür. `drain_to_completed` ile model, geçmişin bir **handoff özeti**ni üretir. Tek client session retry'lar boyunca korunur (sticky routing).

`SessionBudgetExceeded/Interrupted` özel ele alınır. **Örnekte üretilen özet:**

```
## İlerleme
- pytest çalıştı: 47 passed, 3 failed (auth/login testleri)
- auth/login.py okundu; login() 45. satırda çift-doğrulama var
## Kararlar
- token httpOnly cookie'de tutulacak
## Kalan
- login()'i böl, testleri düzelt
```

Adım 8 — (Katman B) Cache-dostu trim (baştan kes)

Ne: Özet history'ye yazılırken, geçmiş **baştan** kırpılır, son mesajlar korunur. Koddaki yorum: *"trim from the beginning to preserve cache (prefix-based) and keep recent messages intact."* **Neden:** Sağlayıcının **prefix cache**'i baştan-aynı mesajlara dayanır. Ortadan/sondan kesmek cache'i bozar; baştan kesmek (eskiyi atıp özetle değiştirmek) en az cache hasarı verir.

Adım 9 — (Katman B) Pencere (window) olarak sakla

Ne: Compaction bir `CompactedItem` üretir:

```
CompactedItem {
  message, // handoff özeti
  replacement_history, // özetin yerini aldığı eski öğeler
  window_number, // pencere sırası
  first_window_id, previous_window_id, // pencere zinciri
}
```

Neden: Pencereleler zincirlenir → eski rollout **resume edilebilir**. Compaction "yok etme" değil, **geri-sarılabılır tarih katmanı**. (Legacy `window_id` şekli bile kabul edilir — geriye uyumluluk.)

Adım 10 — (Resume) summary_prefix ile devam

Ne: Oturum resume edilince özetin önüne `summary_prefix.md` eklenir:

"Another language model started to solve this problem and produced a summary... Use this to build on the work already done and avoid duplicating work." **Neden:** Yeni model, özetin bir **başka modelin** işi olduğunu bilsin ve tekrar etmesin.

Adım 11 — (Kancalar + uzak) Pre/Post + remote

Ne: `run_pre_compact_hooks` / `run_post_compact_hooks` compaction'ı sarar (PreCompact/PostCompact). `compact_remote*.rs` **sunucu-tarafı** compaction sunar (v1/v2) + `compact_model_fallback` (özet modeli düşerse yedek).

Adım 12 — Sonuç

Örnekte SONRA:

```
#0 [system]          "Sen Codex'sin..."          ~2.000 tok   (baştan trim'de korundu
mu? bkz not)
#1 [compactd]        handoff özeti (pytest 47/3, login() 45...) ~200 tok    ← eski #1-#5 yerine
#2 [message user]    "login()'i sadeleştir"          ~30 tok     ← son mesajlar korundu
#3 [message assistant] "Planım: ..."              ~40 tok
+ CompactedItem{window_number:1, ...} kaydı (resume için)
```

~57K → ~2.3K token · handoff özeti + korunan tail · pencere zinciri resume'a hazır

5. Tüm hattın özeti (tek bakış)

Adım	Katman	İşlem	Örnekteki sonuç
1	A	record'da bütçe aş?	#3 (60KB) > 32KB → kes
2	A	truncate_middle	baş+son tut, orta at + Warning
3	A	multimodal koru	#5 görüntü dokunulmaz
4	B	tetik	context eşiği → auto
5	B	context kur	BeforeLastUserMessage
6	B	talimat record	SUMMARIZATION_PROMPT
7	B	MODEL TURN	handoff özeti üret
8	B	baştan trim	prefix cache korunur
9	B	pencere sakla	CompactedItem zinciri
10	resume	summary_prefix	"başka model üretti"
11	B	hooks + remote	Pre/Post + sunucu-tarafı
12	—	sonuç	57K → 2.3K, resume'a hazır

6. Dört sistemle fark

Eksen	Hermes	OpenClaw	OpenCode	Codex
Tool çıktısı	informatif 1-satır	detay şeritle+chunk	spill-to-disk	ortadan-kes (baş+son)
Tarih	orta-turn LLM özet	chunk LLM özet	prune + LLM	model-turn handoff + pencere

Kalıcılık	commit-fence	worker plan	compacted damga	geri-sarılabilir pencere zinciri
Cache	—	—	—	baştan-trim (prefix koru)
Ayırt edici	anti-thrash	güvenlik-şerit	POC'a yakın	windowing + uzak/sunucu + summary_prefix

Öz: Codex tool çıktısını **ortadan keser** (baş+son koru) ve tarih compaction'ını **geri-sarılabilir pencere** olarak saklar — diğerlerinden en farklı iki fikir.

7. POC eşlemesi

Bizim POC (poc/)	Codex karşılığı
fate=KES (kısaltma)	truncate_middle (ama ortadan, baş+son koru)
informatif not	"Warning: truncated output (N tokens)" başlığı
tool_call_id bütünlüğü	record_items + geçerli öge yapısı
SUPERSEDE / tekrar	pencere zinciri (yeni pencere eskiyi replace)
— (bizde yok)	ortadan-kesme (biz baş/son atıyoruz, Codex ortayı) · windowing/resume · baştan-trim cache koruması · uzak/sunucu compaction

Ders: Codex'in **ortadan-kesme** fikri bizde yok — biz genelde kuyruğu kesip başı tutuyoruz; Codex baş+son tutup ortayı atıyor (komut çıktıları için daha akıllı). Ve **windowing** (compaction'ı silmek yerine geri-sarılabilir katman yapmak) tamamen farklı bir felsefe.

Kaynaklar

- ../harnesses/codex/codex-rs/utls/output-truncation/src/lib.rs — truncate_text · formatted_truncate_text · multimodal filtre
- utls/string/src/truncate.rs — truncate_middle_chars · truncate_middle_with_token_budget
- core/src/compact.rs — run_inline_auto_compact_task · run_compact_task · build_compaction_initial_context · baştan-trim
- protocol/src/compacted_item.rs — CompactedItem (window zinciri)
- prompts/templates/compact/{prompt,summary_prefix}.md — handoff özeti + resume prefix
- Eş belgeler: [hermes-tool-trace-compaction.md](#) · [openclaw-tool-trace-compaction.md](#) · [opencode-tool-trace-compaction.md](#) · [poc/](#)

Claude Code — Tool-Trace Compaction: Baştan Sona Tam Rehber (Teknik)

Kaynak notu (dürüstlük): Claude Code'un CLI'ı **kapalı kaynaktır** (minify bundle). Bu belge, sabit isimleri/değerleri iddia etmez; her şey **iki doğrulanabilir kaynağa** dayanır: (1) resmî docs (`code.claude.com/docs` — hooks: PreCompact/PostCompact, context-window, memory) ve (2) **bu oturumda birebir gözlemlenen çalışma-zamanı davranışı** (93KB'lık bir WebFetch çıktısının diske dökülmesi + oturum başındaki auto-compaction özeti). Örnekler gerçek gözlemlerdir; "muhtemelen/belirsiz" işaretli yerler tahmindir.

Akılda-kalıcı "dadı" sürümü için (sonra) bkz. `claude-code-tool-trace-compaction.md`.

0. Kimlik / felsefe — üç katman, iki kaçış yolu

Claude Code tool-trace'i **üç mekanizmayla** yönetir:

- Microcompaction (tool-sonucu düzeyi):** Büyük bir tool çıktısı **diske yazılır**, context'e sadece **referans + önizleme** girer. (Deterministik, LLM'siz — gözlemlendi.)
- Auto-compaction (konuşma düzeyi):** Token eşiğine gelince eski turn'ler bir **konuşma özetine** indirgenir. (LLM'li — gözlemlendi.)
- Subagent izolasyonu (kaçış yolu):** Yan işi *aynı* pencerede sıkıştırmak yerine **ayrı bir context penceresine** (Task tool) taşımak; sadece özet döner.

Yani Claude Code'un iki "kaçış yolu" var: **diske taşı** (microcompaction) veya **ayrı pencereye taşı** (subagent). Sıkıştırma son çare.

1. Sözlük

Terim	Anlamı
context window	Modele gönderilen tüm mesaj yığını için token sınırı.
tool-trace	Tool çağrıları + sonuçlarının context içindeki toplamı.
microcompaction	Tek bir büyük tool çıktısını diske döküp yerine referans bırakma.
disk referansı	Context'te kalan "Full output saved to: ...txt" satırı + kısa önizleme.
auto-compaction	Eşikte tüm konuşmanın bir özete indirgenmesi.
/compact	Kullanıcının elle tetiklediği compaction.
PreCompact / PostCompact	Compaction'ı saran hook olayları (docs).
subagent	Task tool ile açılan izole context; yan işi taşır, özet döndürür.

2. Mekanizmalar (docs + gözlem)

Mekanizma	Kaynak	Ne yapar
Microcompaction	🔍 gözlem	Büyük tool çıktısı → disk dosyası + referans. Bu oturumda 93KB WebFetch bunu tetikledi.
Auto-compaction	🔍 gözlem + docs	Eşikte konuşma özeti. Bu oturum "continued from previous conversation" özetile başlandı.
/compact	📄 docs	Manuel tetik.
PreCompact hook	📄 docs (hooks)	Compaction ÖNCESİ; bloklanabilir.
PostCompact hook	📄 docs (hooks)	Compaction SONRASI; bildirim.
Subagent (Task)	📄 docs + gözlem	Yan işi ayrı pencereye taşır, özet döner.

Sabitler: Kapalı kaynak olduğu için eşik yüzdesi, önizleme boyutu gibi **kesin sayılar bilinmiyor**.
Gözlem: önizleme ~2KB civarıydı, tam çıktı diske yazıldı.

3. Mimari

```
%% Diyagram (mermaid kaynağı):
flowchart TB
    G["tool çıktısı üretildi"] --> M{"Büyük mü?"}
    M -->|evet| MC["Microcompaction: diske yaz + referans/önizleme bırak"]
    M -->|hayır| K["context'te kalır"]
    MC --> T{"Context eşiği aşıldı mı?"}
    T --> PRE["PreCompact hook"]
    PRE --> AC["Auto-compaction: eski turn'ler → konuşma özeti"]
    AC --> POST["PostCompact hook"]
    POST --> C["devam"]
    T -->|hayır| C
    G -. büyük yan-iş .-> SUB["Subagent (Task): ayrı pencere, özet döner"]
```

4. Adım adım — bu oturumdan GERÇEK örneklerle

Adım 1 — Microcompaction: büyük tool çıktısı üretilince (🔍 gözlemlendi)

Ne: Bir tool büyük bir çıktı döndürünce, harness onu **context'e tam koymaz**; **diske yazar**, yerine referans + önizleme bırakır.

Bu oturumdaki GERÇEK örnek — bir `WebFetch` (sub-agents docs) 93.2KB döndürdü. Context'e giren:

```
<persisted-output>
Output too large (93.2KB). Full output saved to:
/home/altan/.claude/projects/.../tool-results/toolu_01BzYj7wFfTyUnU2PQrwWtMR.txt

Preview (first 2KB):
> ## Documentation Index ...
...
</persisted-output>
```

Sonuç: 93.2KB context'e hiç girmedir. Context'te kalan: **~2KB önizleme + disk yolu**. İçeriğe sonradan

ihtiyaç olursa o dosya okunur. **Neden akılda kalsın:** *Dev çıktı context'e değil, diske gider; geriye "adres" kalır.* Bu = **spill-to-disk / referans** (OpenCode'un `truncation-dir`'i, Hermes'in `context_references`'i ile aynı fikir).

Adım 2 — Auto-compaction: context eşliğine gelince (🔍 gözlemlendi + 📄 docs)

Ne: Konuşma çok uzayınca, eski turn'ler tek bir **konuşma özetine** indirgenir; sonra döngü devam eder.

Bu oturumdaki GERÇEK örnek — bu oturum şununla başladı:

```
This session is being continued from a previous conversation that ran out of context.
The summary below covers the earlier portion of the conversation.
Summary: ...
```

Sonuç: Önceki (çok uzun) konuşmanın tamamı yapılandırılmış bir özete indi (Primary Request, Key Concepts, Files, Errors, Pending Tasks...). Yeni pencere bu özetle başladı. **Neden akılda kalsın:** *Konuşma pencereyi doldurunca, geçmiş bir "devir özetine" dönüşür.* (Hermes'in orta-turn özeti, OpenClaw'ın chunk özeti ile aynı aile.)

Adım 3 — PreCompact / PostCompact hook (📄 docs)

Ne: Auto-compaction'ı **kancalar** sarar:

- **PreCompact** — compaction başlamadan; **bloklanabilir** (exit 2 veya `decision:"block"`), context enjekte edebilir.
- **PostCompact** — compaction bittikten sonra; **bildirim** (kontrol yok). **Neden:** Kullanıcı/entegrasyon compaction'a müdahale edebilir (ör. "şu dosyayı özete ekle" veya "şimdi compaction yapma").
Örnek kullanım: Bir `PreCompact` hook'u, önemli bir durum dosyasını `additionalContext` ile özete ekleyebilir.

Adım 4 — /compact (manuel tetik, 📄 docs)

Ne: Kullanıcı `/compact` yazarak compaction'ı **elle** tetikler (eşliği beklemeden). **Neden:** Kullanıcı "şimdi temizle, uzun bir işe girişeceğim" diyebilirsin.

Adım 5 — Subagent kaçış yolu (Task tool, 📄 docs + gözlem)

Ne: Büyük bir yan-iş (arama, log tarama, çok dosya okuma) ana pencereyi şişirecekse, harness onu **subagent'a** delege eder: subagent **ayrı bir context penceresinde** çalışır, ara adımları ana pencereye **hiç girmez**, sadece **özet** döner. **Neden:** Bu, compaction'a bir **alternatif**dir: aynı pencerede sıkıştırmak yerine, kirli detayı hiç o pencereye sokmamak. **Örnek:** "Şu 40 dosyayı tara ve login akışını özetle" → subagent 40 dosyayı kendi penceresinde okur, ana pencereye tek paragraf özet döner. 40 dosyanın içeriği ana context'e hiç girmez. **Neden akılda kalsın:** *Kirli işi ayrı odada yap, ana odaya sadece özeti getir.* (OpenClaw'ın "arka oda"sıyla benzer ruh, ama orada temizlik; burada işin kendisi ayrı pencerede.)

5. Tüm hattın özeti (tek bakış)

Katman	Tetik	Yöntem	Kaynak	Örnek (bu oturum)
Microcompaction	büyük tool çıktısı	diske yaz + referans	🔍 gözlem	93KB WebFetch → disk + 2KB önizleme

Auto-compaction	context eşiği	konuşma özeti	 + 	oturum başı "continued from..." özeti
Hook	compaction anı	Pre/Post müdahale	 docs	PreCompact blokla/enjekte
Manuel	<code>/compact</code>	elle tetik	 docs	—
Subagent	büyük yan-ış	ayrı pencere, özet döner	 + 	40 dosya tarama → tek özet

6. Dört sistemle fark

Eksen	Hermes	OpenClaw	OpenCode	Codex	Claude Code
Tool çıktısı	informatif satır	detaş şeritle+chunk	disk spill	ortadan-kes	disk referansı (microcompaction)
Konuşma	orta-turn LLM özet	chunk LLM özet	prune+LLM	model-turn+pencere	auto-compaction (konuşma özeti)
Kaçış yolu	—	—	—	windowing	subagent (ayrı pencere)
Kanca	—	—	—	Pre/Post	PreCompact/PostCompact
Kaynak	açık	açık	açık	açık	kapalı (docs+gözlem)
Ayırt edici	anti-thrash	worker plan	POC'a yakın	ortadan-kes+resume	iki kaçış: diske ya da ayrı pencereye taşı

Öz: Claude Code, tool-trace'i sıkıştırmaktan çok "**taşır**": dev çıktıyı diske (microcompaction) ya da yan-ış ayrı pencereye (subagent) taşır; ancak son çare olarak konuşmayı özetler (auto-compaction). Kapalı kaynak olduğu için sabitleri bilinmez ama davranışı bu oturumda birebir gözlemlendi.

7. POC eşlemesi

Bizim POC (poc/)	Claude Code karşılığı
Referansa indirme (spill)	Microcompaction (disk referansı) — birebir aynı fikir
Konuşma özeti	Auto-compaction
Fayda-freni / eşik	(belirsiz — kapalı kaynak)
<code>tool_call_id</code> bütünlüğü	(harness dahil — gözlemlenemedi)
— (bizde yok)	subagent kaçış yolu (ayrı pencere) · Pre/PostCompact hook ekosistemi

Not: Claude Code, bizim POC'un "referansa indirme" fikrini (microcompaction) ve konuşma özetini paylaşır; ama en güçlü fikri **subagent kaçış yolu** — tool-trace'i sıkıştırmak yerine hiç o pencereye sokmamak. Bu, "compaction"dan önce gelen bir context-yönetim stratejisidir.

Kaynaklar

-  code.claude.com/docs — hooks (PreCompact/PostCompact), context-window, sub-agents, memory.
-  Bu oturumun çalışma-zamanı gözlemi — 93KB WebFetch microcompaction'ı (`tool-results/...txt`)

referansı), oturum başı auto-compaction özeti.

- Karşılaştırma: [hermes-tool-trace-compaction.md](#) · [openclaw-tool-trace-compaction-teknik.md](#) · [opencode-tool-trace-compaction-teknik.md](#) · [codex-tool-trace-compaction-teknik.md](#) · [poc/](#)